Name: Vidit Bansal
Roll no: 2401420018

Course: B.Tech CSE (Data science) IBM
Sem: 4
Section: A

Page No:
Date: / /

Assigment - 1

~~Probability & Statistics~~

Analysis & Design of Algorithm

Q1 a) $T(n) = T(n-1) + C$
$= T(n-2) + 2C$
$= T(n-3) + 3C$
$\vdots$
$= T(0) + nC$

$T(0) = C$

Time Complexity $\rightarrow T(n) = C + nC = O(n)$

Space Complexity $\rightarrow O(n)$

no. of calls = n

b)

Step counting	Frequency
i = 0	1
i < n	n + 1
arr[i] == key	n
break	1
return	1

Best Case
Comparisons = 1

$T(n) = 1$
Best $= O(1)$

worst case:
$T(n) = n$
worst $= O(n)$

average case
$T(n) = \frac{n}{2}$

Average $= O(n)$

space complexity

$O(1)$

c) (i) Recurrence:

$T(n) = T(n-1) + c$

Same as factorial $\quad O(n)$

(ii) Recurrence : $T(n) = 2T(n-1) + c$

Expand: $T(n) = 2(2T(n-2))$
$= 4T(n-2)$
$= 2^n T(0)$
$O(2^n)$

Q2

Case	Meaning	Asymptotic
Best	Min time	$\Omega(f(n))$
Average	Expected time	$\Theta(f(n))$
Worst	Max time	$O(f(n))$

worst case dominates because algorithm must always be prepared for worst input.

Q3 Insertion Sort.
Elements : 5, 2, 4, 6, 1, 3

Pass 1 → insert 2 before 5
2 5 4 6 1 3

Pass 2 → insert 4 between 2 and 5
2 4 5 6 1 3

Pass 3 → 6 already correct
2 4 5 6 1 3

Pass 4 → insert 1 at beginning
1 2 4 5 6 3

Page No:
Date: / /

Pass 5 → insert 3 between 2 and 4.
1 2 3 4 5 6

Sorted output → 1, 2, 3, 4, 5, 6

Time Complexity

Best	Already Sorted	$O(n)$
Average	Random order	$O(n^2)$
Worst	Reverse order	$O(n^2)$

Space Complexity → $O(1)$

Q4 Recursion Tree
$$T(n) = T(n/3) + T(2n/3) + Cn$$
level 0

Cn

level 1

$C(n/3) + C(2n/3) = Cn$

level 2

Cn

each level costs Cn
height of Tree

$\log n$

Total cost

$T(n) = Cn \log n$

$O(n \log n)$

Page No:
Date: / /

Q5 Recursion Tree

$T(n) = 3T(\frac{n}{4}) + n^2$

level 0

n^2

level 1

$3(\frac{n}{4})^2 = 3n^2/16$

level 2

$3^2(n/16)^2$

Series decreases geometrically

using Master Theorem:

$a = 3, b = 4$

$n^{\log_4 3} \approx n^{0.79}$

Since,

$n^2 > n^{0.79}$ $T(n) = O(n^2)$

Q6 $T(n) = aT(n/b) = f(n)$

Case I

if: $f(n) < n^{\log_b a}$

$T(n) = \Theta(n^{\log_b a})$

Case II

if: $f(n) = n^{\log_b a}$

$T(n) = \Theta(n^{\log_b a} \log n)$

Case III

if: $f(n) > n^{\log_b a}$

$T(n) = \Theta(f(n))$